

STAT 341: Assignment 2

DUE: Friday, June 20, 2025 by 11:59pm EST

NOTES

Your assignment must be submitted by the due date listed at the top of this document, and it must be submitted electronically in .pdf format via Crowdmark. This means that your responses for different questions should begin on separate pages of your .pdf file. Note that your .pdf solution file must have been generated by R Markdown. Additionally:

- For mathematical questions: your solutions must be produced by LaTeX (from within R Markdown). Neither screenshots nor scanned/photographed handwritten solutions will be accepted – these will receive zero points.
- For computational questions: R code should always be included in your solution (via code chunks in R Markdown). If code is required and you provide none, you will receive zero points.
- For interpretation questions: plain text (within R Markdown) is required. Text responses embedded as comments within code chunks will not be accepted.

Organization and comprehensibility is part of a full solution. Consequently, points will be deducted for solutions that are not organized and incomprehensible. Furthermore, if you submit your assignment to Crowdmark, but you do so incorrectly in any way (e.g., you upload your Question 2 solution in the Question 1 box), you will receive a 5% deduction (i.e., 5% of the assignment's point total will be deducted from your point total).

QUESTION 1 (10 marks): Loaded Dice and the non-centrality parameter

The deviation from expectation, $\sum (E - O)^2 / E$ of a fair d -sided die follows a standard χ^2 distribution with $d - 1$ degrees of freedom, where the expected count is $1/d$ for each side. (For example, on a standard 6-sided die, we fit a chi-squared with 5 degrees of freedom)

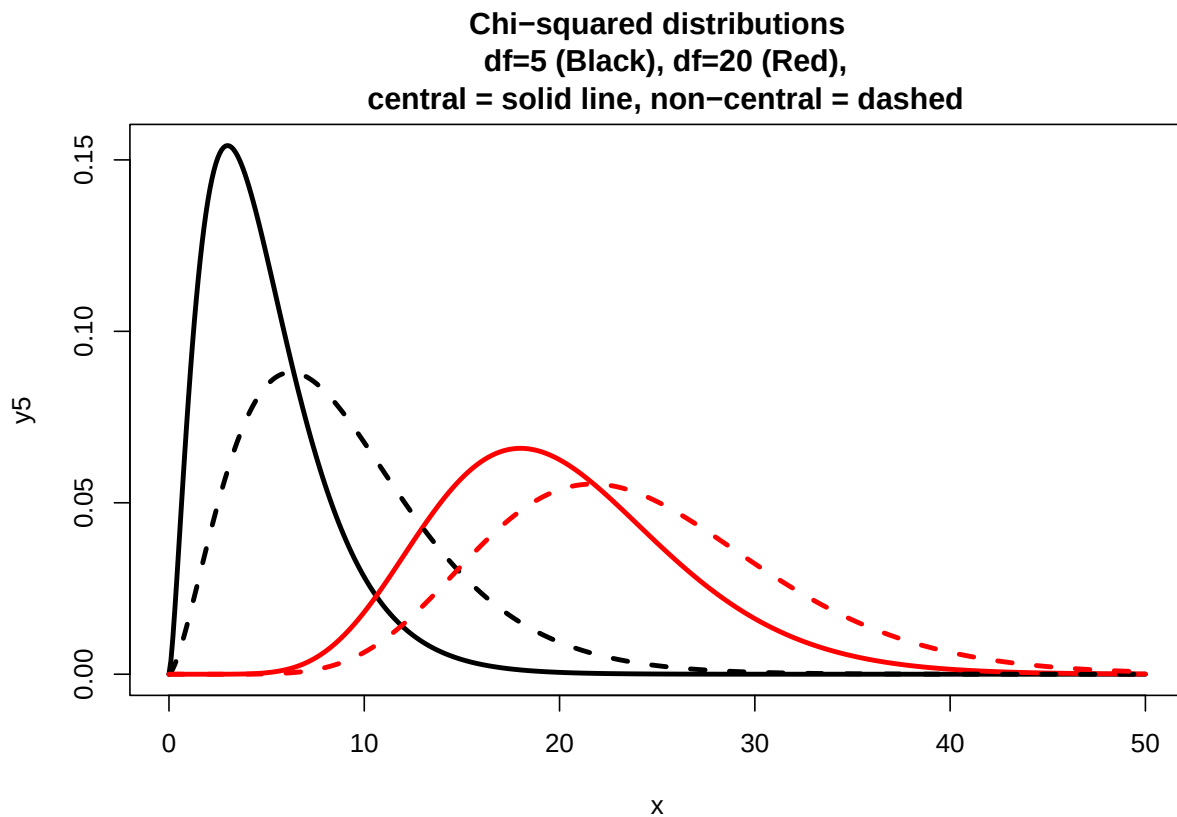
However, a die that is UNFAIR will deviate from the expected fair values according to the **non-central** chi-squared. That is, where a chi-squared distribution has a non-centrality parameter $\theta > 0$.

Below is a plot of the chi-squared distribution with 5 and 20 degrees of freedom and a non-centrality parameter θ of 0, and 4.

```
x = seq(0, 50, 0.1)
y5 = dchisq(x, df=5)
y20 = dchisq(x, df=20)

y5_nc = dchisq(x, df=5, ncp=4)
y20_nc = dchisq(x, df=20, ncp=4)

plot(x, y5, lwd=3, type="l",
      main="Chi-squared distributions
            df=5 (Black), df=20 (Red),
            central = solid line, non-central = dashed")
lines(x, y20, lwd=3, lty=1, col="Red")
lines(x, y5_nc, lwd=3, lty=2, col="Black")
lines(x, y20_nc, lwd=3, lty=2, col="Red")
```



- a) (5 marks) Create a function `rho_ncp_fn = function(x=0, rollset)` to find the negative log-likelihood of 12000 independent rolls of a fair die, and three different unfair dice. The arguments it takes in are `x`, the non-centrality parameter, and `rollset`, the set of rolls that are generated in the following code that makes `rolls_fair` and so on. You can use `dchisq()` instead of writing out the PDF.

Unfair die 1

$$p_1 = \frac{1}{12}, \quad p_2 = \frac{2}{12}, \quad p_3 = \frac{2}{12}, \quad p_4 = \frac{2}{12}, \quad p_5 = \frac{2}{12}, \quad p_6 = \frac{3}{12}$$

Unfair die 2

$$p_1 = \frac{3}{24}, \quad p_2 = \frac{4}{24}, \quad p_3 = \frac{4}{24}, \quad p_4 = \frac{4}{24}, \quad p_5 = \frac{4}{24}, \quad p_6 = \frac{5}{24}$$

Unfair die 3

$$p_1 = \frac{3}{12}, \quad p_2 = \frac{3}{12}, \quad p_3 = \frac{3}{12}, \quad p_4 = \frac{1}{12}, \quad p_5 = \frac{1}{12}, \quad p_6 = \frac{1}{12}$$

```
# Makes the rollset
set.seed(341)
rolls_fair = sample(1:6, size=12000, replace=TRUE)
rolls_unfair1 = sample(1:6, size=12000, replace=TRUE,
  prob=c(1,2,2,2,2,3)/12)
rolls_unfair2 = sample(1:6, size=12000, replace=TRUE,
  prob=c(3,4,4,4,4,5)/24)
rolls_unfair3 = sample(1:6, size=12000, replace=TRUE,
  prob=c(3,3,3,1,1,1)/12)
```

- b) (2 marks) Use `optim(... method="Brent", lower=0, upper=10)` and the rho function you made in part b to find the maximum likelihood estimates of the function.
- c) (3 marks) The true value θ of the non-centrality parameter per observation is

$$\theta = \sum_{i=1}^p \frac{(a_i - b_i)^2}{b_i}$$

where a_i is the true probability of that event happening (e.g., the unfair die roll probabilities), and b_i is the assumed probability (e.g., $1/6$ because we assume all dice are fair)

Find the θ for each of the three unfair dice, and compare your answers to what `optim()` has provided.

Source: https://en.wikipedia.org/wiki/Noncentral_chi-squared_distribution

QUESTION 2 (15 marks): Dog Agility Part 2 - Optimizing a Bezier Curve

This is part 2 of the project to analyze maps of dog agility courses and assess them for things like overall difficulty. In this first part, we're just going to be establishing the functions necessary to determine the physical dynamics of an arbitrary path.

In this part, we're going to use generalized optimization and gradient descent in order to estimate a likely path that a dog would take from one obstacle to another by optimizing cubic Bezier curves.

A cubic Bezier curve is defined by a starting point, an ending point and two guide points that are used for interpolation.

For example, the Bezier curve we examined in assignment 1 has a starting point of (,) an ending point of (,) and guide points of (,) and (,).

- a) (3 marks) Given the following starting point, ending point, and second guide point (point2). Calculate the `rho` for each of the NA values left behind by the data frame made by this code. Report the `head()` and the `tail()` of `df_1a` for marking.

```
# Makes the data frame to use
x = rep(seq(0,10,0.1), times=101)
y = rep(seq(0,10,0.1), each=101)
df_1a = data.frame(x,y)
df_1a$rho = NA

head(df_1a)
```

```
##      x y rho
## 1 0.0 0  NA
## 2 0.1 0  NA
## 3 0.2 0  NA
## 4 0.3 0  NA
## 5 0.4 0  NA
## 6 0.5 0  NA
```

The easiest way to do this is to run a `for(k in 1:nrow(df_1a))` loop where `df_1a$rho[k]` is calculated by putting `c(df_1a$x[k], df_1a$y[k])` into `x` for `make_bezier_path_1free`, and putting the resulting `$x` and `$y` into `get_path_badness`.

```
make_bezier_path_1free = function(x=c(3,5), point2=c(7,5), begin=c(0,0), end=c(10,2))
{
  px = c(begin[1], x[1], point2[1], end[1])
  py = c(begin[2], x[2], point2[2], end[2])

  t = seq(from=0, to=1, by=0.01)
  bezier_path_x = (1-t)^3*px[1] +
    3*(1-t)^2*t*px[2] +
    3*(1-t)*t^2*px[3] +
    t^3*px[4]
  bezier_path_y = (1-t)^3*py[1] +
    3*(1-t)^2*t*py[2] +
    3*(1-t)*t^2*py[3] +
    t^3*py[4]

  return(list(x=bezier_path_x, y=bezier_path_y))
}
```

Our badness/loss function is

$$\rho(\mathbf{x}, \mathbf{y}) = a \times \max(\text{acceleration})^2 + b \times (\text{distance})^2 + c \times \max(\text{direction change})^2$$

Where the values $a = 10$, $b = 1$, and $c = 20$ would ideally come from expert opinion such a dog agility judge, or a vet, but in this case they are arbitrarily made up.

The squared factor is to prevent any one factor from being maximized order to bring the other factors to zero. This function is calculated for a given $(\mathbf{x}, \mathbf{y})$ with the following function:

```
get_path_badness = function(path,
                             factor_acc=10, factor_dist=1, factor_cdir=20)
{
  x = path$x
  y = path$y
  dx = diff(x)
  dy = diff(y)
  d2x = c(diff(dx), 0)
  d2y = c(diff(dy), 0)

  # [[ACCELERATION]]
  acc = sqrt(d2x^2 + d2y^2)

  # [[DISTANCE]]
  ds = sqrt(dx^2 + dy^2)

  # [[CHANGE IN DIRECTION]]
  dir = atan(dy/dx)
  ddir = abs(diff(dir))

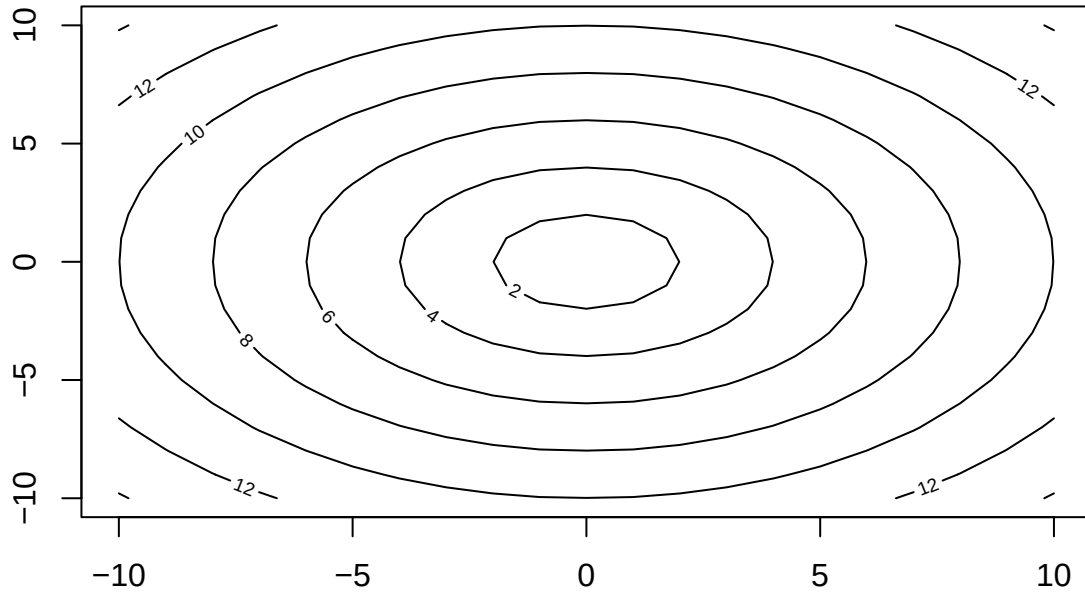
  badness = factor_acc * max(acc)^2 +
            factor_dist * sum(ds)^2 +
            factor_cdir * max(ddir)^2

  return(badness)
}
```

- b) (2 marks) Draw a contour plot of the response surface, where the response is ‘badness’, and the surface is on the x and y coordinates of the second guide point. Use the following code as an example. (Hint: You can convert `df$rho` into a `matrix()` of 101 rows and columns)

```
# x is a 21-length vector
# y is a 21-length vector
# z is x^2 + y^2. z is a 21x21 matrix
x <- -10:10
y <- -10:10
z <- sqrt(outer(x ^ 2, y ^ 2, "+"))

contour(x, y, z)
```



c) (3 marks) Fit a third order polynomial with interactions to the response surface.

Namely, use the linear least squares method to find the coefficient estimates $\hat{\beta}$ to, the following formula.

$$\rho = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 y + \beta_5 y^2 + \beta_6 y^3 + \beta_7 xy + \beta_8 x^2 y + \beta_9 xy^2 + \epsilon$$

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

Where ω is the estimated badness, where X is `cbind()` of 1, `df_1a$x`, This should be 10 columns (one for each beta value) and 101^2 rows (one for each coordinate), and where Y is the vector of 101^2 badness values.

You may use `lm()`, specifically `summary(lm())$coef` to check your work, but your code should include the matrix multiplications of X and Y. Hints: `solve(X)` gives the inverse of X, `t(X)` gives the transpose, and `X %*% Y` gives the matrix multiplication of X and Y.

d) (2 marks) Write a function named `rho_fn` which takes in `x=c(x0, y0)`, where `x0` and `y0` are the coordinates of the first guide point in the Bezier curve. `rho_fn` should do two things:

- Generate the path object from `x` by calling a function.
- Calculate ρ , the badness of that path, by calling function.
- Return ρ .

e) (1 mark) Use `optim(...,method="L-BFGS-B", lower=c(0,0), upper=(10,10))` and the `rho_fn` function you made in part d to find

$$\arg \min_{0 \leq x \leq 10, 0 \leq y \leq 10} \rho(x, y)$$

- f) (2 marks) Extend the function `make_bezier_path_1free` into `make_bezier_path_2free`, in which `x` takes in 4 values, the first two being `x` and `y` of the first guide point, and the second two being `x` and `y` of the second guide point, respectively.
- g) (2 marks) Make a new `rho_fn2` based on `rho_fn`, but for `make_bezier_path_2free`, and use `optim` with it to find and report the best location to put both guide points.

QUESTION 3 (16 marks): Quantile Regression

Part 0) Either run the following code to get the census data on every subdivision in Canada, or use the dataset in Learn, if you're having issues with your API.

```
library(cancensus)

#all_vecs = list_census_vectors("CA21")
# Total English      v_CA21_2209
# Total French       v_CA21_2212
# Full time Full year v_CA21_6525
# Average wks worked v_CA21_6531
# Median after-tax inc v_CA21_986
# Median Total inc    v_CA21_983

all_regions = list_census_regions("CA21") # 2021 census
csd_all = list(CSD = all_regions$region)
all_regions = data.frame(all_regions)

thislevel = "CSD"
census_data = get_census(dataset = "CA21",
                        regions = csd_all,
                        vectors = c("v_CA21_2209",
                                    "v_CA21_2212",
                                    "v_CA21_983",
                                    "v_CA21_986",
                                    "v_CA21_6525",
                                    "v_CA21_6531"),
                        level = thislevel)

names(census_data)[ncol(census_data) - 5 ] = "English_Speakers"
names(census_data)[ncol(census_data) - 4 ] = "French_Speakers"
names(census_data)[ncol(census_data) - 3 ] = "Median_Before_Tax"
names(census_data)[ncol(census_data) - 2 ] = "Median_After_Tax"
names(census_data)[ncol(census_data) - 1 ] = "FTime_FYear_Workers"
names(census_data)[ncol(census_data) - 0 ] = "Avg_Weeks_Worked"

census_data = data.frame(census_data)
census_data = subset(census_data, Population > 0 & !is.na(Avg_Weeks_Worked)
                    & !is.na(Median_After_Tax))
filename = paste0("Stat341_S25_A2_Census_Data_",thislevel, ".csv")
write.csv(census_data, filename, row.names=FALSE)
```

- (5 marks) Create a function called `rho_lm_fn(x, data)`, where `x` is a 2x1 vector of the intercept (α) and slope (β) respectively, which returns the sum of squares of the residuals.
- (2 marks) Use the `optim()` function to find the α and β of the least squares solution. Use `lm()$coef` and compare your results.
- (3 marks) Create a function called `rho_qr_fn(x, tau, data)`, where `x` is a 2x1 vector of intercept (α) and slope (β) respectively, which returns the sum of the generalized absolute value loss where `tau` is the target quantile.

- d) (4 marks) Use the `optim()` function to find the alpha and beta of the generalized absolute loss solutions for $\tau = 0.1, 0.25, 0.5, 0.75, 0.9$. Use `rq()`\$coef function in to the `quantreg` package to check your work.
- e) (2 marks) Compare your results to the `rq` results, do they all agree? Do they mostly agree? What does this imply about the stability of a generalized optimizer like `optim` compared to an application-specific optimizer like `rq` or `lm`?